Mistral Use Case Take-Home

Wendy Aw

Table of contents

Problem Statement

Problem statement

Over 600,000 US patent applications are filed each year, and manually classifying these high-volume, multi-label filings into CPC codes is slow, inconsistent, and costly.

Context

- **CPC = Cooperative Patent Classification**
- A system for classifying patents into categories based on their technical content.
- Developed and maintained by the European Patent Office (EPO) and the United States Patent and Trademark Office (USPTO)
- Used worldwide to organize patents consistently across countries

The Users

Patent office examiners

Patent attorneys

R&D teams

The Challenge

1. High volume & complexity

2. Multi-label nature

3. Specialized language & jargon

4. Manual classification is slow, error-prone, requires expertise

Business Impact

- **Faster filing and time-to-grant**
- **Accelerated IP research and innovation cycles**
  - uncover white-space
  - spot emerging trends in new filings
  - benchmark competitor activity in same classes
- **Cost savings**
  - reduce manual review hours
  - reduce classification errors and appeals
- **Scalability**
  - automated system can handle spikes in filing volume – no need to hire extra experts

Current landscape

- **USPTO's AI classification tool**
- **EPO (European Patent Office) CPC Text Categoriser**
- **Commercial & API Solutions**
    - Platforms like PatSnap and Derwent integrate proprietary classifiers via REST APIs for bulk portfolio analysis and visualization.

Why LLMs?

Broad world knowledge
Pretrained on large diverse datasets, they understand context, facts and terminology across many domains

Deep language understanding
LLMs excel at parsing long, complex texts, capturing nuanced technical context

Domain-specialized language training
Can be further trained to understand specialized jargon across diverse domains

Fast scalable classification
Can process high volumes of filings in parallel

Data Preparation

Dataset Overview

- Source: USPTO PatentsView, PatentSearch API
- Year: 2024
- Downloaded 85,000 granted patents
- Fields extracted:
  - patent_id
  - patent_grant_year
  - patent_type
  - patent_abstract
  - cpc_current (contains patent's assigned classes, subclasses and groups)
  - description_text
  - description_length

CPC Classification System

8 SECTIONS

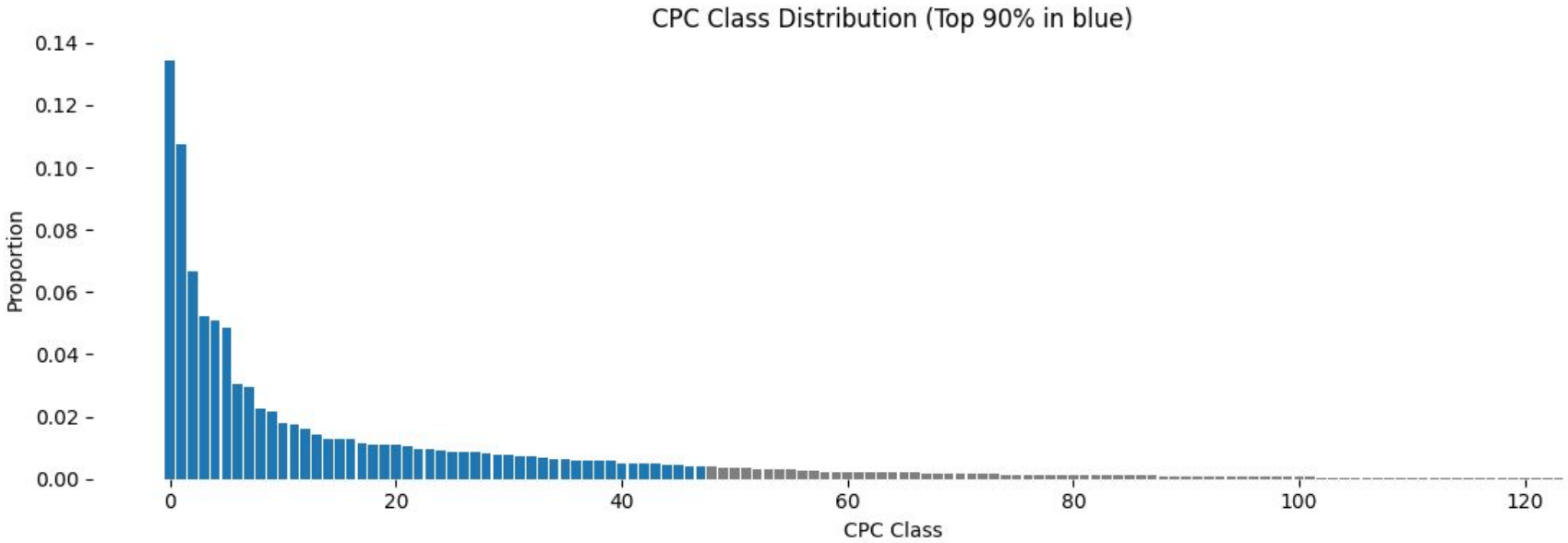
A : Human Necessities
B : Performing Operations; Transporting
C : Chemistry; Metallurgy
D : Textiles; Paper
E : Fixed Constructions
F : Mechanical Engineering; Lighting; Heating; Weapons; Blasting
G : Physics
H : Electricity
Y : General Tagging of New Technological Developments…

137 CLASSES

A01: Agriculture; forestry…
A21: Baking; edible doughs
A22: Butchering; meat treatment…
…
H01: Electric elements
H03: Electronic circuitry
…

SUBCLASSES & GROUPS

A01B: Soil working in agriculture or forestry..
A01B 1/02 Spades and shovels
A01B1/06 Hoes, hand cultivators
A01C: Planting; sowing; fertilising
A01C3/00 Treating manure
A01C7/00 Sowing
H01B: Cables; conductors; insulators…
H01C: Resistors
H01F: Magnets; inductances; transformers…
…



Dataset of 85,000 samples captured 129 classes, with 90% of samples belonging only to top 48 classes

Data Pipeline

Label sampling

Sample from 48 most frequent CPC classes, ensuring we train on labels that cover 90% of the data

Patent text truncation

To capture richer technical context beyond the abstract, but stay within the model's input limit, we truncate each document to its first 5,000 words and strip out table artifacts where necessary.

Train/validation/test split

Generate reproducible splits with two strategies.

Format transformation

Transform raw records into Mistral's fine-tuning format ready for direct ingestion by the API

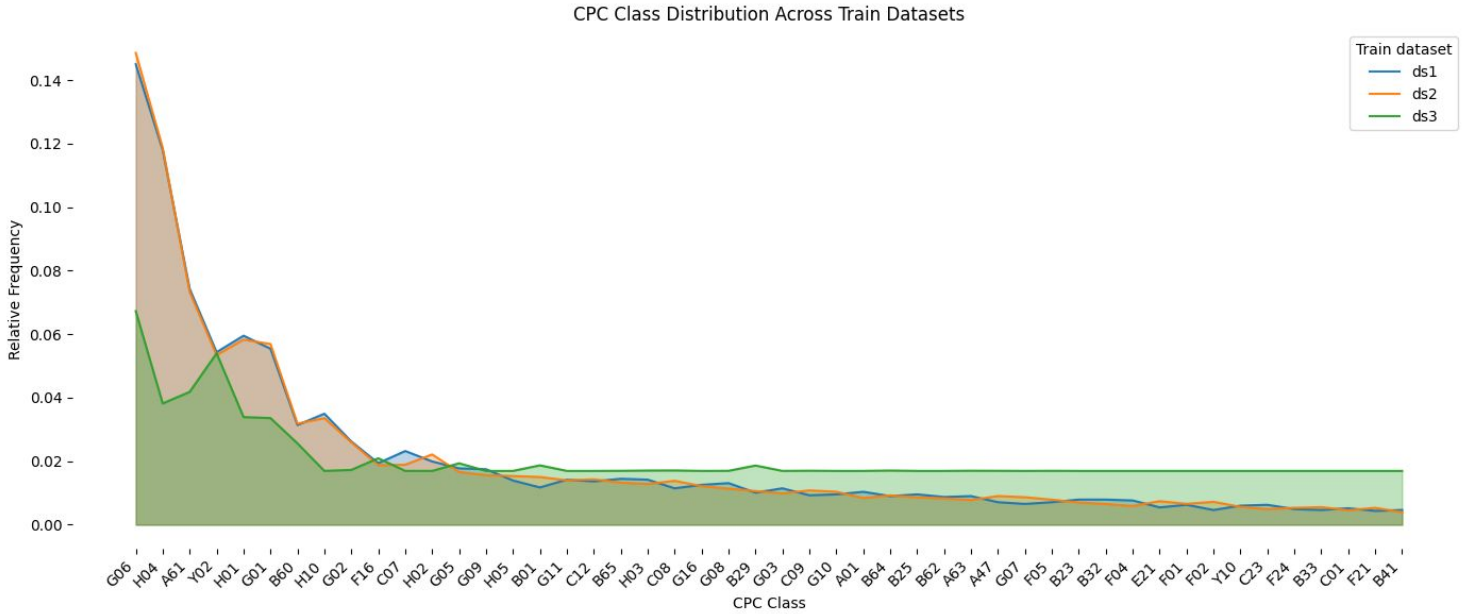
Dataset Split

Strategy 1: Multilabel stratified sampling (test, validation, ds1, ds2)
Preserves the original distribution of each class across train, test, validation sets

Strategy 2: Balanced sampling (ds3)
At least 600 samples per class, prioritizing rarest classes first
Ensures almost equal representation across all 48 CPC classes

Dataset	Size
Test	228
Validation	251
Train ds1	2,041
Train ds2	11,083
Train ds3	15,147

Dataset Split



Measuring Success

Multi-label metrics

▲ Subset accuracy (very strict)
Fraction of samples where predicted class set exactly matches true class set
How often did we get all classes exactly right for a sample?

▼ Hamming loss
Fraction of individual class predictions that are incorrect, across all classes and all samples
What proportion of individual class decisions were wrong overall?
(but must interpret with caution since we have many classes but few positive labels)

▲ Jaccard index
Ratio of correctly predicted classes to total unique classes
How much did the predicted and true class sets overlap on average?

▲ Higher is better
▼ Lower is better

Performance indicators

▲ Micro precision
Overall fraction of all predicted classes that were actually correct across every class decision.
How accurate were our positive guesses when considering every class decision equally?

▲ Macro precision
The average precision across classes, treating each class equally no matter its size.
How accurate were our positive guesses on average for each class?

▲Micro recall
Overall fraction of all actual positives that we correctly identified across every class decision.
How good were we at finding true classes when counting every class decision equally?

▲Macro recall
The average recall across classes, treating each class equally no matter its frequency.
How good were we at finding true classes on average for each class?

▲ Higher is better
▼ Lower is better

Performance indicators

▲ Micro F1
A single score that balances overall precision and recall across every class decision.
How well did we balance catching true classes and avoiding false alarms for all classes combined?

▲ Macro F1
The average of F1 scores computed separately for each class, giving each class equal importance.
How well did we perform on each class on average, regardless of how common they are?

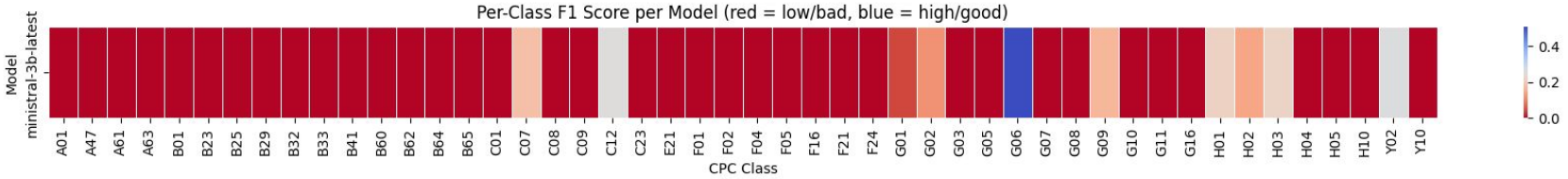
▼ Hallucination rate
Fraction of predicted classes that turned out to be invalid.
How often did the LLM invent classes that weren't actually possible?

▲ Higher is better
▼ Lower is better

Per-class Analysis

▲Per-class F1 score
Identifies which classes the model handles well vs poorly

Weak class identification
List of classes with F1 score < 0.5



Baseline Performance

Testing Baseline Performance

Models tested: ministral-3b / ministral-8b / mistral-small / mistral-medium

Test set: 228 patent descriptions covering all top 48 classes

Establish a clear performance floor
– Quantifies the minimum accuracy we'd expect for the task
– Serves as a baseline for comparing all subsequent, larger models and techniques

Evaluate out-of-the-box capabilities
– Measures model performance without any fine-tuning overhead
– Reveals how much "free" accuracy we'd get just by prompting the base models

Testing Baseline Performance

Consistent Evaluation

- Same prompts across all models for fair comparison

Batch Processing

- Used Mistral's batch API for time and cost efficient large-scale evaluation

Prompt Engineering Strategies

Zero-shot
Asking the model to perform a task by providing only the instructions

Few-shot
Giving the model a few example input–output pairs alongside the instructions, so it can infer the desired format and perform the task on new inputs

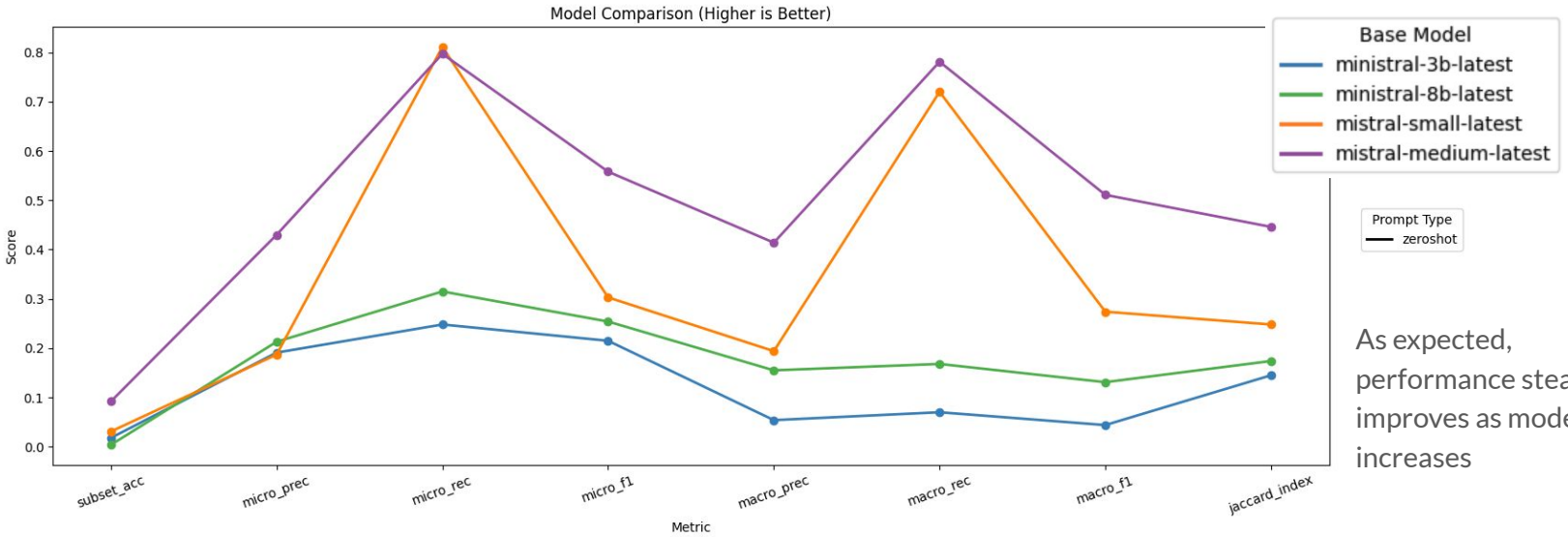
Chain-of-Thought (CoT)
Encouraging the model to generate and expose its intermediate reasoning steps before arriving at a final answer

Prompt template (Zero-shot)

```
SYSTEM PROMPT

You are an expert patent examiner specializing in the Cooperative Patent
Classification (CPC) system.

Your task is to review a given patent application and identify all CPC
**class-level** codes (e.g., "G06", "A61") that are relevant to the
subject matter described.

You will be provided with:
- The full text of the patent application.
- A dictionary of CPC class codes and their descriptions.

Instructions:
- Carefully analyze the technical content of the patent application.
- Select all relevant **class-level** CPC codes based on the invention's
core features and purpose.
- Ignore subclasses, groups, or specific notations—only output the
applicable **CPC class IDs**.
- Return the result as a JSON array of class IDs. Example: ["Y02", "H01"]
```

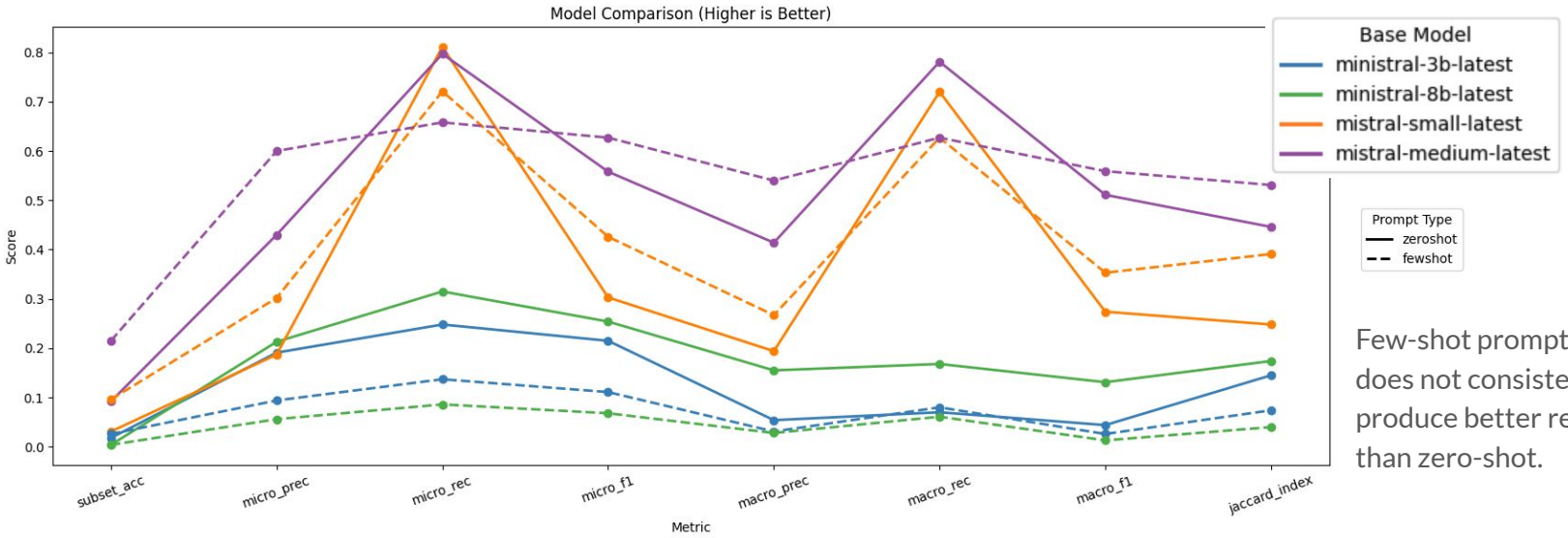
```
USER PROMPT

# CPC Class IDs
## A : Human Necessities
- A01: AGRICULTURE; FORESTRY; ANIMAL
HUSBANDRY; HUNTING; TRAPPING; FISHING
… (truncated list of 48 class definitions)
- Y10: TECHNICAL SUBJECTS COVERED BY FORMER
USPC

# Patent description
{patent_description}

# Task
Assign the CPC class IDs to the patent
description. Return the result formatted as
a JSON array of class IDs. Example: ["Y02",
"H01"]. Do not include any other text or
backticks in the response.
```

Base Model Comparison (Zero-shot)



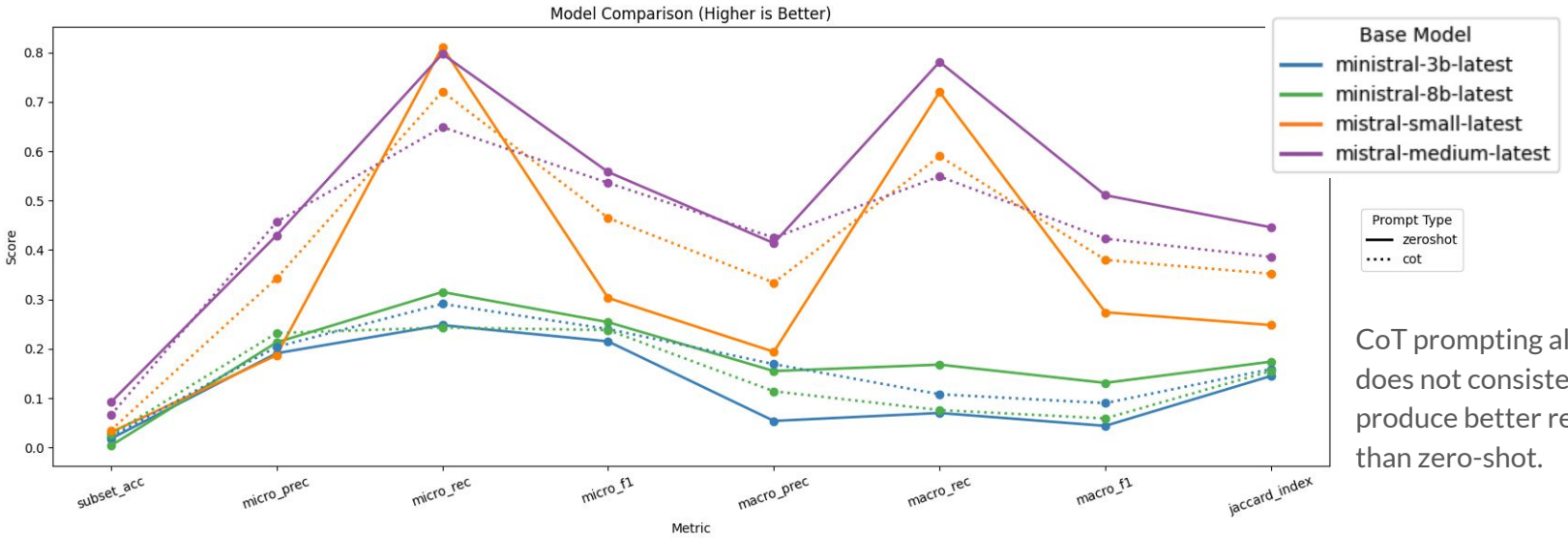
As expected, performance steadily improves as model size increases

Base Model Comparison (Few-shot)



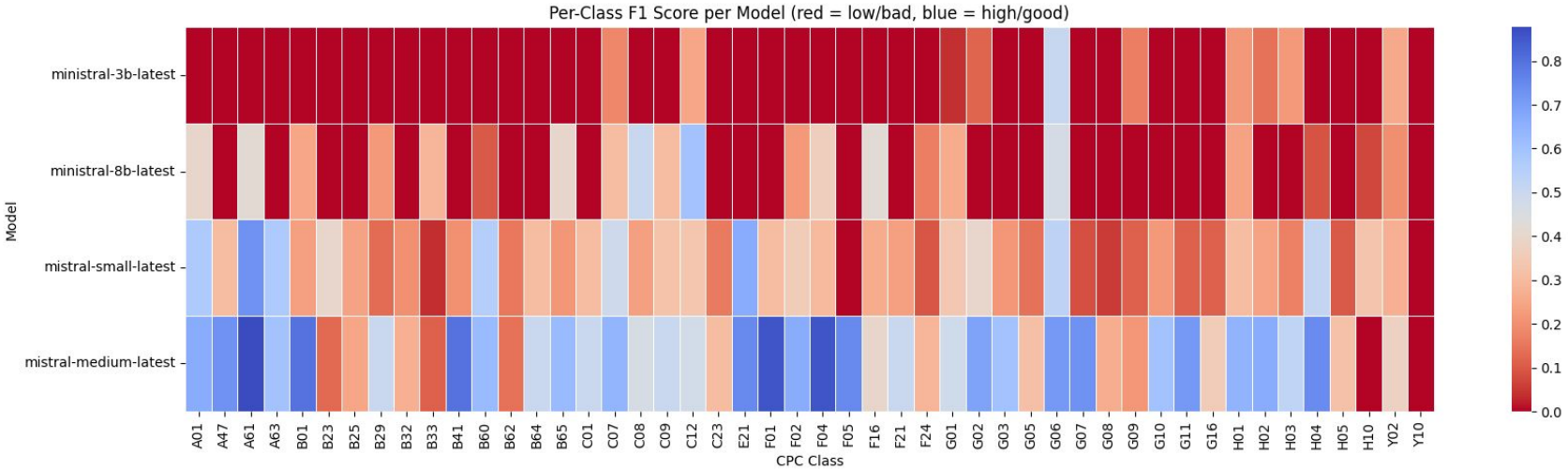
Few-shot prompting does not consistently produce better results than zero-shot.

Base Model Comparison (CoT)



CoT prompting also does not consistently produce better results than zero-shot.

Base Model Comparison (Zero-shot)



Why Fine-tuning?

Baseline performance gaps

1. Improvable baseline scores
2. Prompt limitation: complex rules don't fit
3. Inconsistent output formats

Cost efficiency

1. No prompt overhead
2. Smaller model

Performance benefits

1. Faster predictions
2. Better format consistency
3. Domain specialization
4. Hallucination reduction

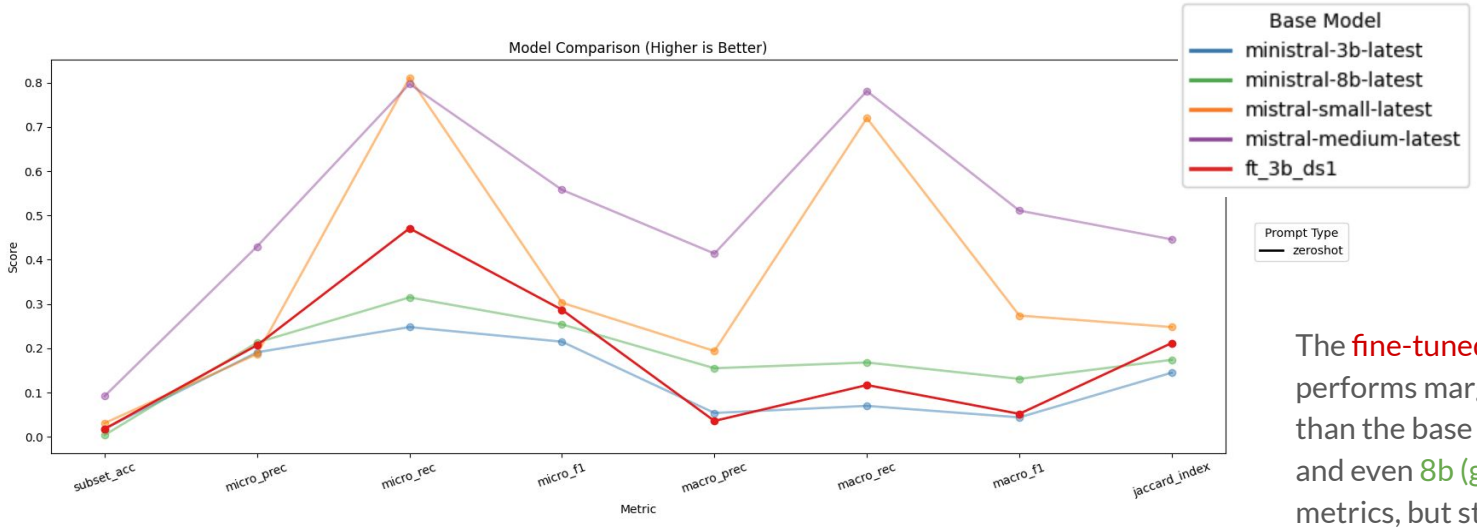
Fine-tuning

Fine-tuning Decisions

```
epochs: 1
learning_rate: 0.00001 (or 1e-5)
job_type: classifier
model: ministral-3b-latest
```

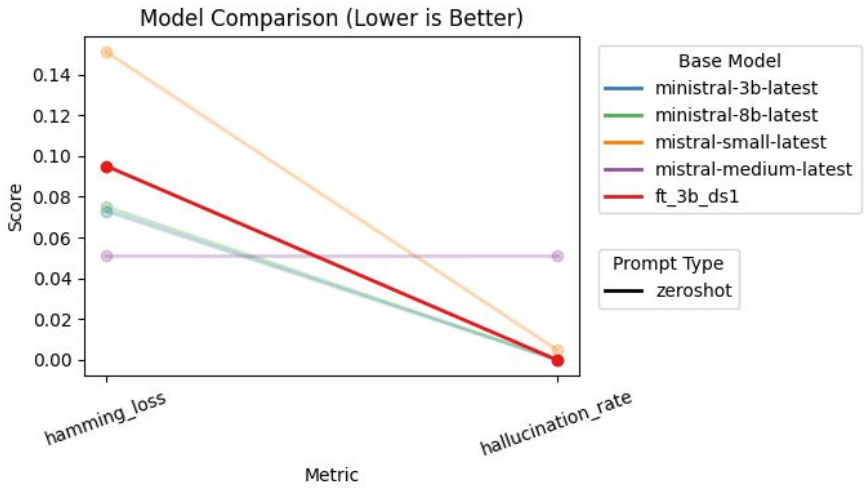
Dataset	Size	Class distribution	Fine-tuning status
Train ds1	2,041	Matching original	Completed
Train ds2	11,083	Matching original	Stalled halfway and remains running
Train ds3	15,147	Balanced	Remains queued and not running

Test Performance: ds1



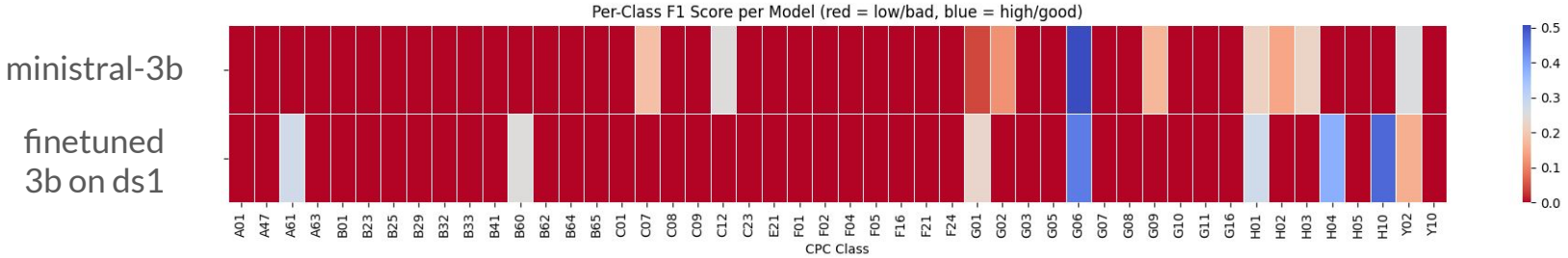
The fine-tuned model (red) performs marginally better than the base 3b model (blue), and even 8b (green) on some metrics, but still falls short of the other larger models.

Test Performance: ds1



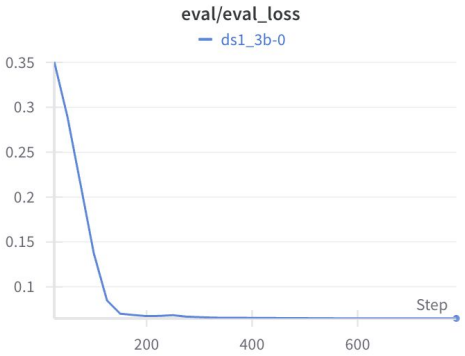
Hamming loss and hallucination rate remain low, which is reassuring. Our model isn't over-predicting classes or fabricating labels.

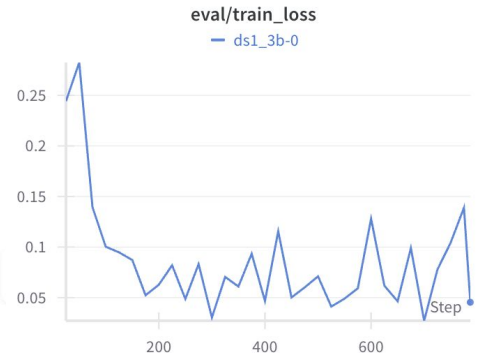
Test Performance: ds1



The 8 (non-red) classes with notably higher F1 scores for ft_3b_ds1 coincide with the top 8 most frequent CPC labels in the training set—highlighting that the model performs best on well-represented categories and underscoring the need to augment data for long-tail classes to elevate their performance.

Training performance: ds1





Early plateauing of both train and validation loss indicates the model has converged and has learned all it can under the current configuration. We would need to explore:

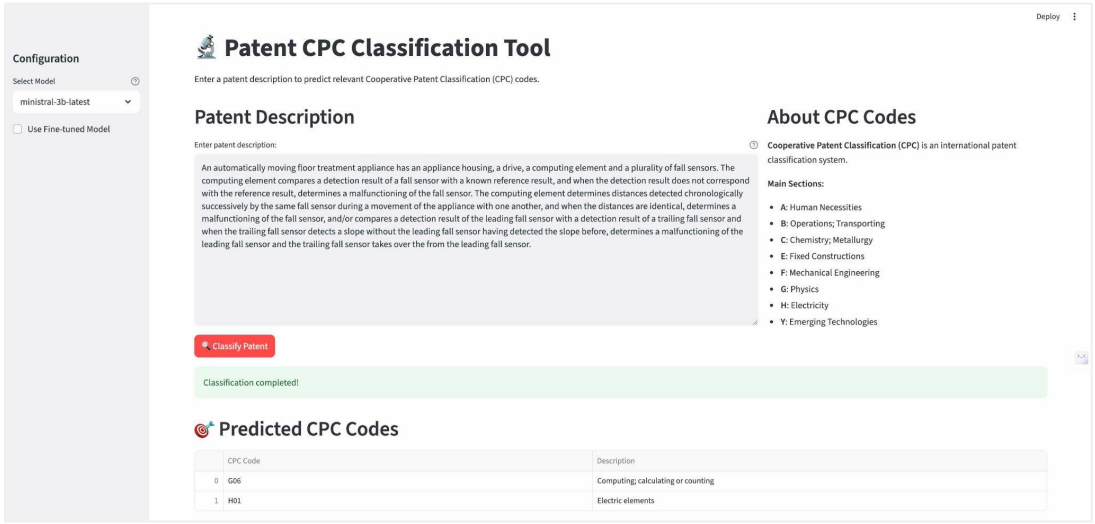
- More or more diverse data (especially under-represented classes)
- Hyperparameter adjustments (learning rate schedules to escape local minimums)
- Increased model size

Training Performance: ds2 (more data)

Although the full ds2 training run wasn't complete, the early consistent gains across evaluation metrics indicate that the **larger dataset ds2 (yellow) is delivering higher overall accuracy** and substantially better coverage of under-represented classes compared to ds1 (red).

Demo

Simple UI demo




```
streamlit run patent_classifier_ui.py
```

Built with Streamlit and Claude Code

Future Enhancements

Future enhancements

- **Data Improvements**
  - Expand Dataset: Curate additional patent records to address long-tail classes
  - Balanced Sampling: Implement advanced sampling strategies for rare class coverage
- **Training Optimizations**
  - LoRA Fine-tuning: Switch to more cost/time-efficient parameter-efficient methods
  - Larger Models: Experiment with 8B+ models once LoRA enables affordable training
  - Early Stopping: Monitor validation metrics to prevent overfitting or learning saturation
- **Technical Scaling**
  - Full CPC Hierarchy: Extend beyond 48 classes to complete classification system
  - Multilingual Support: Add patent classification for non-English documents

Areas of improvement for developer experience

Issue	Suggestion
Fine-tuning jobs remain QUEUED/RUNNING for hours with no update on progress.	Hook into the fine-tuning job's internal stages (queuing, data prep, training, finalizing) and expose percentage-complete or ETA in the UI. Surface a warning banner or send a notification when jobs exceed an expected duration, so users know something's wrong without checking constantly. If a job is idle, let users cancel and immediately re-queue it with one click—no need to navigate away or craft a new request.
Had to figure out through trial and error that only ministral-3b could be fine-tuned with the Classifier Factory `{"detail": "Model not allowed for classifier finetuning"}`	Clearly state the allowed models in the error message and prominently in the documentation, not just in the FAQ.

Areas of improvement for developer experience

Issue	Suggestion
Seemingly unable to run more fine-tuning jobs even when there's sufficient balance in the wallet. `{"detail": "Launching a fine tuning job requires credits in your wallet. You can add credits via the console. Current monthly usage: 83.32 EUR. Job cost: 46.36 EUR. Wallet balance: 100.21 EUR."}`	Clarify or explicitly show "amount reserved for running/queued jobs," "Used this month," and "Credits remaining" so users understand why sufficient wallet balance may still block new jobs.
~16 K token inputs failed validation, forcing trial and error to find a size that passes.	When an input exceeds the token threshold, return a precise error message that states both the offending input size and the maximum supported token limit—so users immediately know how much to trim without guesswork.
Unable to cancel 'Validated' jobs from the UI. Had to do it via an API request.	Expose the existing API's cancel endpoint in the UI for "Validated" jobs—so users can click "Cancel" directly.
Completions from batch inference sometimes struggle to output valid JSON.	It would be helpful to support enabling JSON mode for batch inference.